

Báo cáo bài tập thực hành tuần 6

Họ và tên: Nguyễn Văn Phúc Huy
MSSV: 23110163
Lớp: 23TTH (Chiều thứ 6, ca 1)

```
In [43]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from matplotlib.colors import ListedColormap
```

Thực thi đoạn code ở mục 1

Đọc dữ liệu từ file

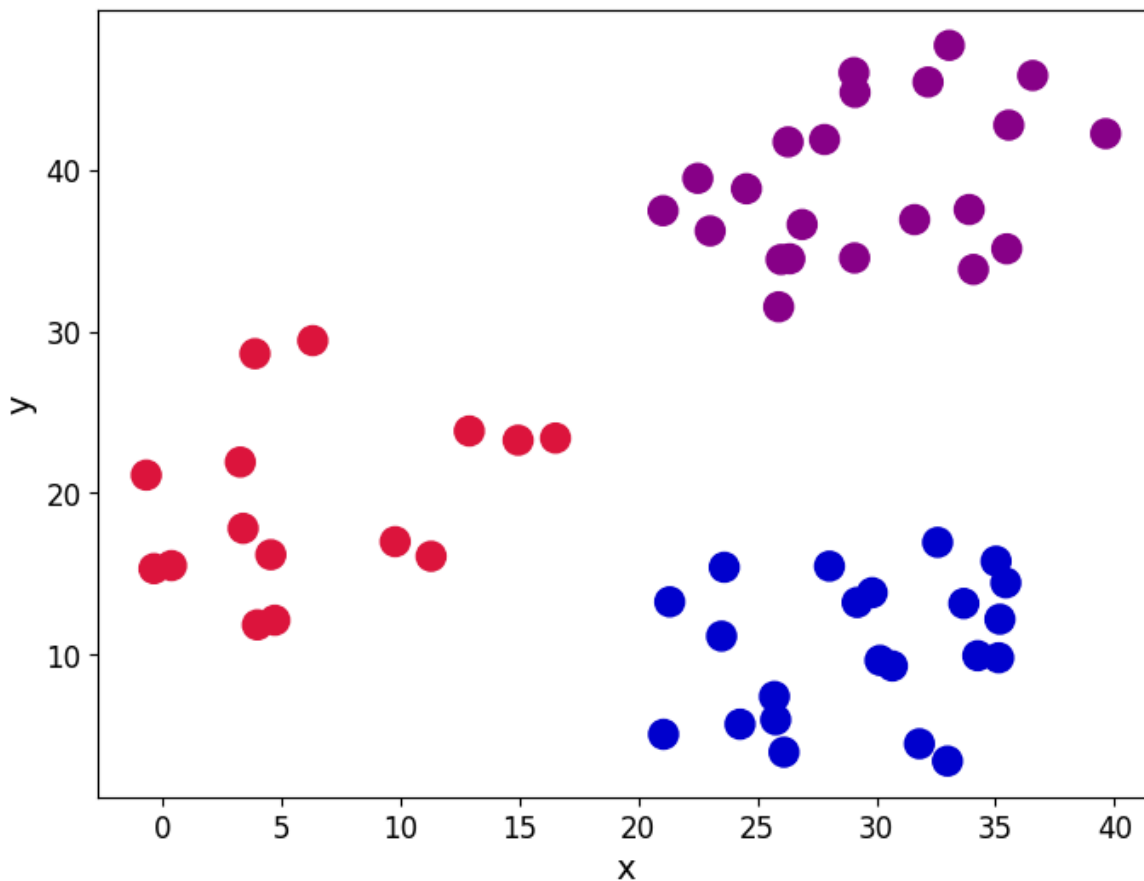
```
In [44]: blobs = pd.read_csv(r'D:\Data-Mining_MTH10358\Homeworks\TH\6\data.csv')
colnames = list(blobs.columns[1:-1])
print(blobs.head())
```

	ID	x	y	cluster
0	1	35.190	12.189	1
1	2	26.288	41.718	2
2	3	0.376	15.506	0
3	4	26.116	3.963	1
4	5	25.893	31.515	2

Chúng ta sẽ sử dụng cột phân cụm để hiển thị các nhóm khác nhau được biểu diễn trong tập dữ liệu

```
In [45]: customcmap = ListedColormap(["crimson", "mediumblue", "darkmagenta"])

fig, ax = plt.subplots(figsize=(8, 6))
plt.scatter(x=blobs['x'], y=blobs['y'], s=150,
            c=blobs['cluster'].astype('category'),
            cmap = customcmap)
ax.set_xlabel(r'x', fontsize=14)
ax.set_ylabel(r'y', fontsize=14)
plt.xticks(fontsize=12)
plt.yticks(fontsize=12)
plt.show()
```



Bước 1 và 2: Xác định k và khởi tạo các tâm

```
In [46]: def initiate_centroids(k, dset):
...
    Select k data points as centroids
    k: number of centroids
    dset: pandas dataframe
...
    centroids = dset.sample(k)
    return centroids

np.random.seed(42)
k=3
df = blobs[['x','y']]
centroids = initiate_centroids(k, df)
print(centroids)
```

```
      x      y
0  35.190  12.189
5  23.606  15.402
34 23.492  11.142
```

Bước 3: tính khoảng cách

```
In [47]: def rsserr(a,b):
...
    Calculate the root of sum of squared errors.
    a and b are numpy arrays
...
    return np.sum(np.square(a-b))

for i, centroid in enumerate(range(centroids.shape[0])):
    err = rsserr(centroids.iloc[centroid,:], df.iloc[34,:])
    print('Error for centroid {0}: {1:.2f}'.format(i, err))
```

```
Error for centroid 0: 137.94
Error for centroid 1: 18.16
Error for centroid 2: 0.00
```

Bước 4: gán giá trị các tâm

```
In [48]: def centroid_assignment(dset, centroids):
...
    Given a dataframe `dset` and a set of `centroids`, we assign each
```

```

data point in `dset` to a centroid.
- dset - pandas dataframe with observations
- centroids - pandas dataframe with centroids
...

k = centroids.shape[0]
n = dset.shape[0]
assignment = []
assign_errors = []

for obs in range(n):
    # Estimate error
    all_errors = np.array([])
    for centroid in range(k):
        err = rsserr(centroids.iloc[centroid, :], dset.iloc[obs,:])
        all_errors = np.append(all_errors, err)

    # Get the nearest centroid and the error
    nearest_centroid = np.where(all_errors==np.amin(all_errors))[0].tolist()[0]
    nearest_centroid_error = np.amin(all_errors)

    # Add values to corresponding lists
    assignment.append(nearest_centroid)
    assign_errors.append(nearest_centroid_error)

return assignment, assign_errors

```

```

In [49]: df.loc[:, 'centroid'], df.loc[:, 'error'] = centroid_assignment(df, centroids)
print(df.head())

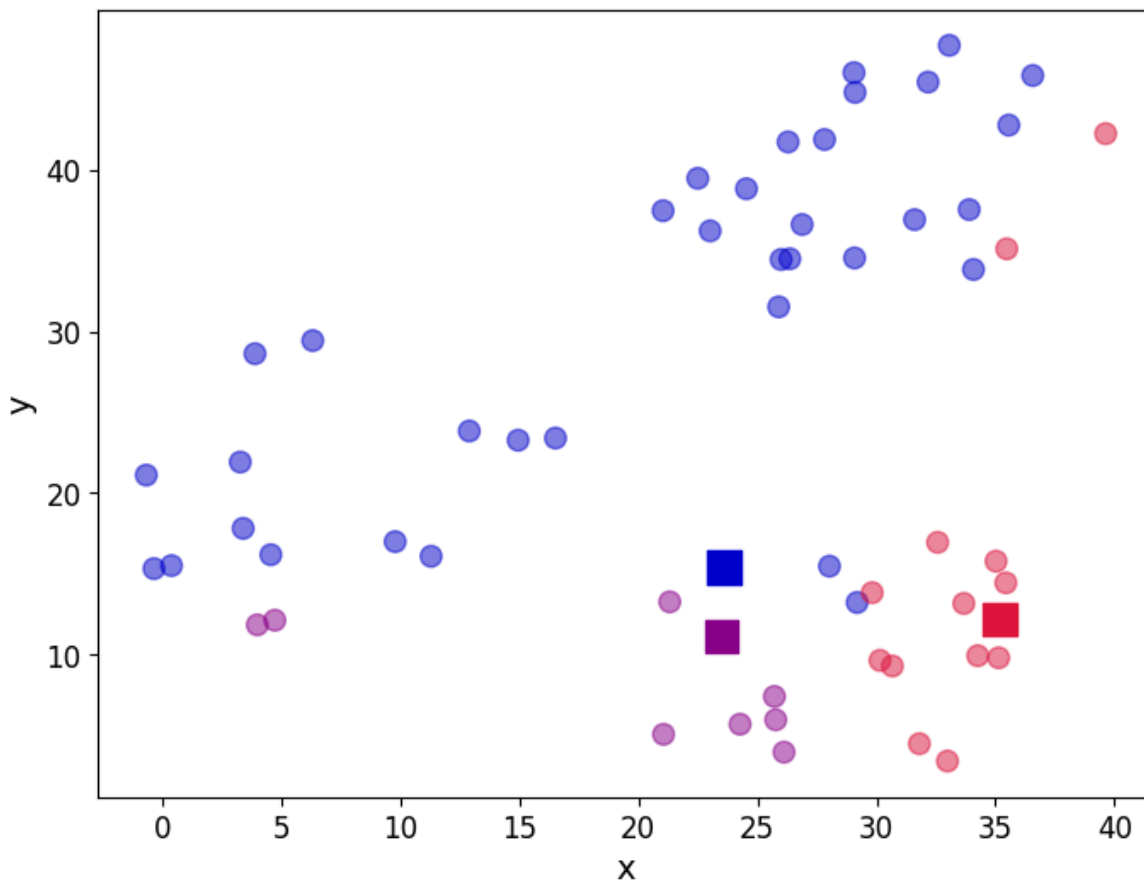
```

	x	y	centroid	error
0	35.190	12.189	0	0.000000
1	26.288	41.718	1	699.724980
2	0.376	15.506	1	539.643716
3	26.116	3.963	2	58.423417
4	25.893	31.515	1	264.859138

```

In [50]: fig, ax = plt.subplots(figsize=(8, 6))
plt.scatter(df.iloc[:,0], df.iloc[:,1], marker = 'o',
            c=df['centroid'].astype('category'),
            cmap = customcmap, s=80, alpha=0.5)
plt.scatter(centroids.iloc[:,0], centroids.iloc[:,1],
            marker = 's', s=200, c=[0, 1, 2],
            cmap = customcmap)
ax.set_xlabel(r'x', fontsize=14)
ax.set_ylabel(r'y', fontsize=14)
plt.xticks(fontsize=12)
plt.yticks(fontsize=12)
plt.show()

```



```
In [51]: print("total error: {0:.2f}".format(df['error'].sum()))
```

total error: 20408.22

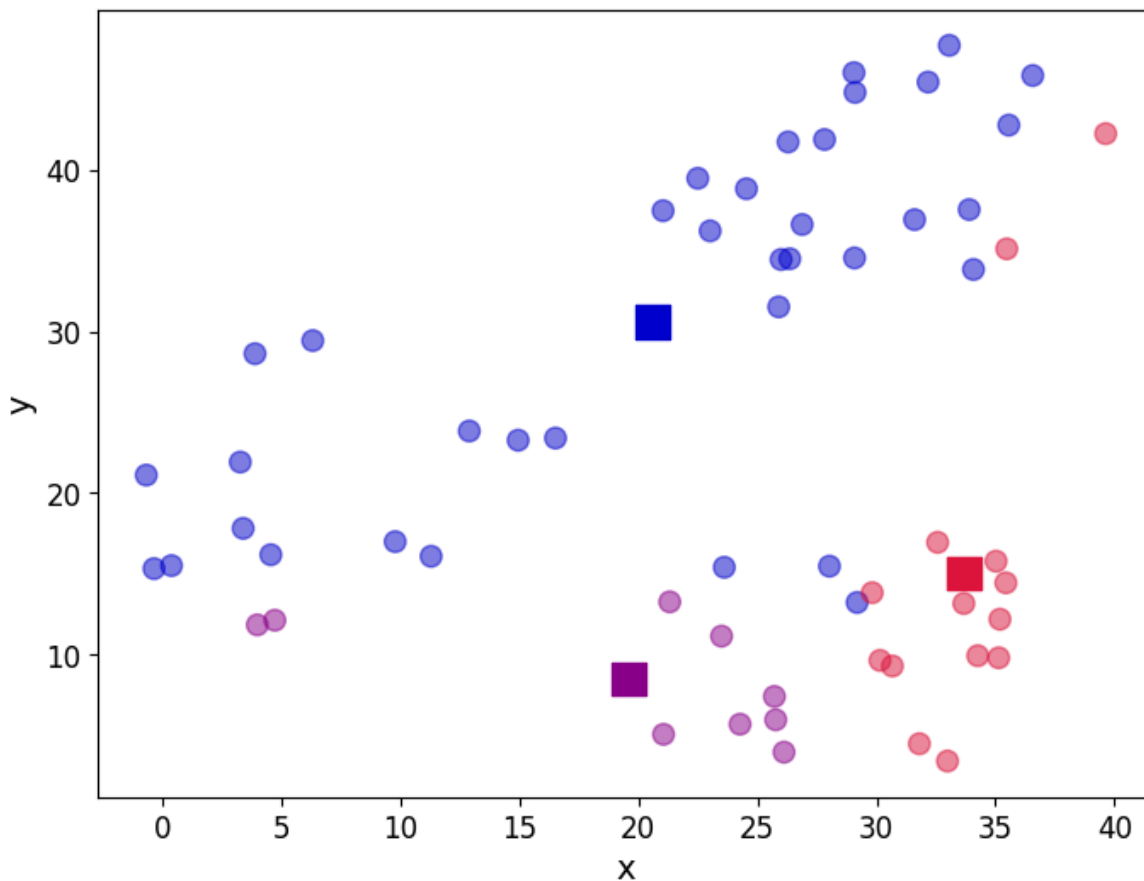
Bước 5: cập nhật vị trí của k tâm bằng việc tính giá trị trung bình của các quan sát được gán cho mỗi tâm

```
In [52]: centroids = df.groupby('centroid').agg('mean').loc[:, colnames].reset_index(drop=True)
print(centroids)
```

	x	y
0	33.703214	15.017429
1	20.595944	30.595361
2	19.602778	8.496556

Xem lại biểu đồ phân tán với vị trí của k tâm đã được cập nhật

```
In [53]: fig, ax = plt.subplots(figsize=(8, 6))
plt.scatter(df.iloc[:,0], df.iloc[:,1], marker = 'o',
            c=df['centroid'].astype('category'),
            cmap = customcmap, s=80, alpha=0.5)
plt.scatter(centroids.iloc[:,0], centroids.iloc[:,1],
            marker = 's', s=200,
            c=[0, 1, 2], cmap = customcmap)
ax.set_xlabel(r'x', fontsize=14)
ax.set_ylabel(r'y', fontsize=14)
plt.xticks(fontsize=12)
plt.yticks(fontsize=12)
plt.show()
```



Bước 6: lặp lại bước 3-5

```
In [54]: def kmeans(dset, k=2, tol=1e-4):
...
    K-means implementationd for a
    `dset`: DataFrame with observations
    `k`: number of clusters, default k=2
    `tol`: tolerance=1E-4
    ...

    # Let us work in a copy, so we don't mess the original
    working_dset = dset.copy()
    # We define some variables to hold the error, the
    # stopping signal and a counter for the iterations
    err = []
    goahead = True
    j = 0

    # Step 2: Initiate clusters by defining centroids
    centroids = initiate_centroids(k, dset)

    while(goahead):
        # Step 3 and 4 - Assign centroids and calculate error
        working_dset['centroid'], j_err = centroid_assigantion(working_dset, centroids)
        err.append(sum(j_err))

        # Step 5 - Update centroid position
        centroids = working_dset.groupby('centroid').agg('mean').reset_index(drop = True)

        # Step 6 - Restart the iteration
        if j>0:
            # Is the error less than a tolerance (1E-4)
            if err[j-1]-err[j]<=tol:
                goahead = False
            j+=1

        working_dset['centroid'], j_err = centroid_assigantion(working_dset, centroids)
        centroids = working_dset.groupby('centroid').agg('mean').reset_index(drop = True)
    return working_dset['centroid'], j_err, centroids
```

Thực thi hàm trên

```
In [55]: np.random.seed(42)
df['centroid'], df['error'], centroids = kmeans(df.iloc[:, :-2], k=3, tol=1e-4)
```

```
print(df.head())
```

	x	y	centroid	error
0	35.190	12.189	0	37.415091
1	26.288	41.718	1	16.201232
2	0.376	15.506	2	51.798518
3	26.116	3.963	0	52.188602
4	25.893	31.515	1	74.265774

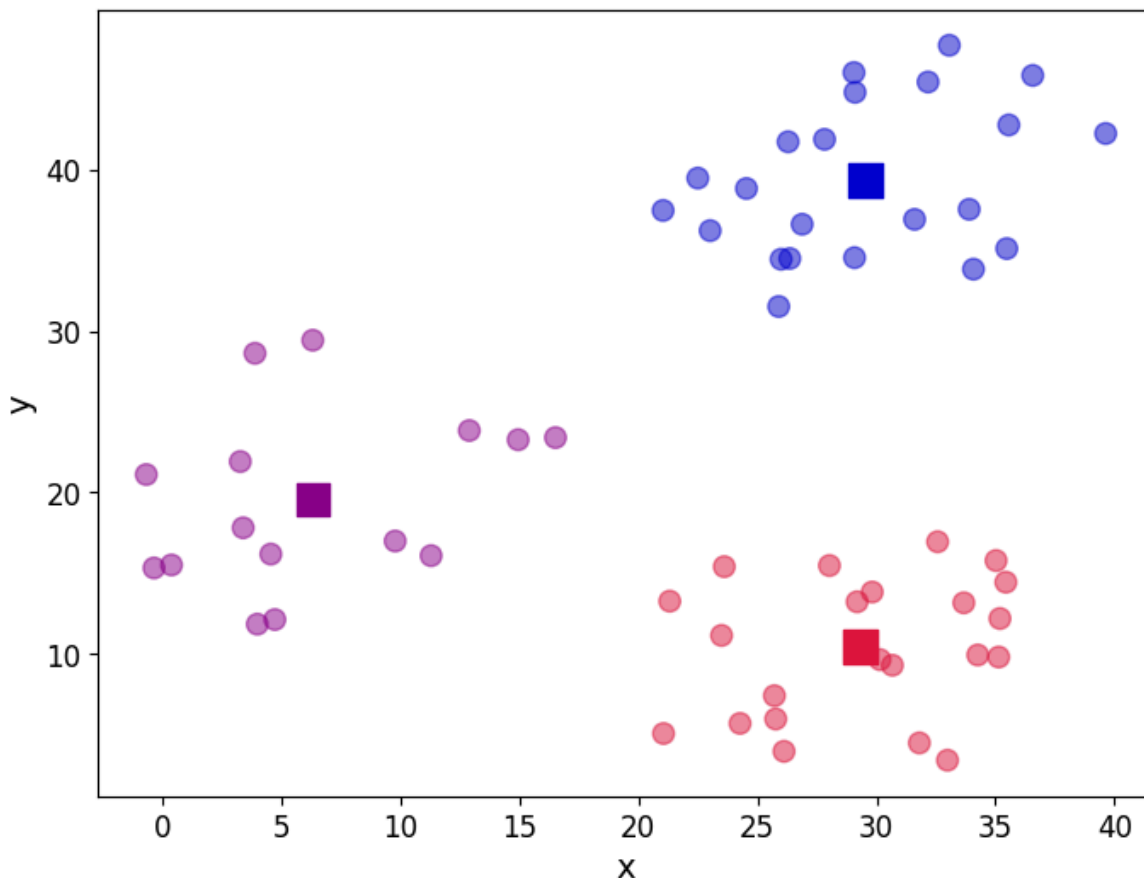
Vị trí của các tâm cuối cùng

```
In [56]: print(centroids)
```

	x	y
0	29.330864	10.432409
1	29.527364	39.328909
2	6.322867	19.559800

Xem lại biểu đồ phân tán

```
In [57]: fig, ax = plt.subplots(figsize=(8, 6))
plt.scatter(df.iloc[:,0], df.iloc[:,1], marker = 'o',
            c=df['centroid'].astype('category'),
            cmap = customcmap, s=80, alpha=0.5)
plt.scatter(centroids.iloc[:,0], centroids.iloc[:,1],
            marker = 's', s=200, c=[0, 1, 2],
            cmap = customcmap)
ax.set_xlabel(r'x', fontsize=14)
ax.set_ylabel(r'y', fontsize=14)
plt.xticks(fontsize=12)
plt.yticks(fontsize=12)
plt.show()
```



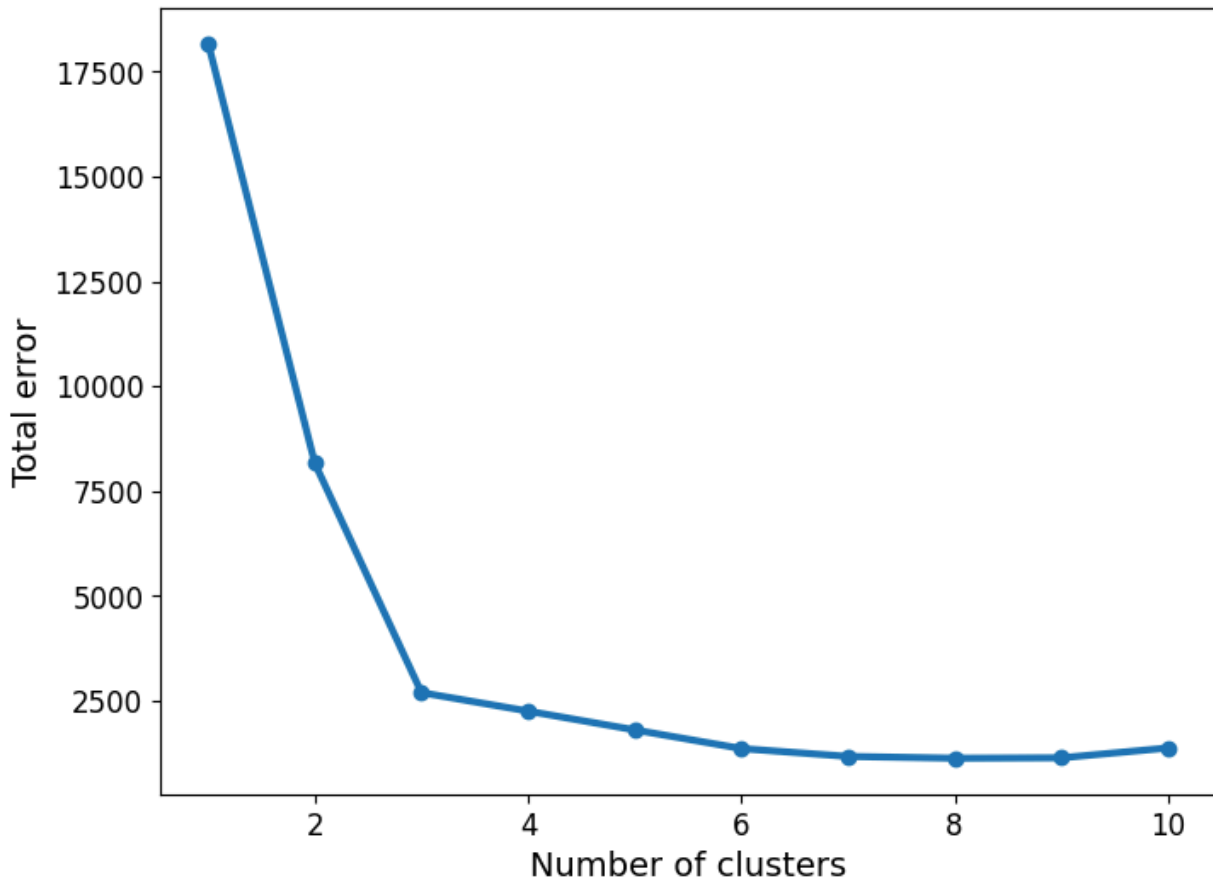
Sử dụng "elbow" để chỉ ra số cụm tối ưu

```
In [58]: err_total = []
n = 10

df_elbow = blobs[['x', 'y']]

for i in range(n):
    _, my_errs, _ = kmeans(df_elbow, i+1)
    err_total.append(sum(my_errs))
fig, ax = plt.subplots(figsize=(8, 6))
plt.plot(range(1, n+1), err_total, linewidth=3, marker='o')
```

```
ax.set_xlabel(r'Number of clusters', fontsize=14)
ax.set_ylabel(r'Total error', fontsize=14)
plt.xticks(fontsize=12)
plt.yticks(fontsize=12)
plt.show()
```



Thuật toán K-medians.

Hàm `plot_cluster` được sử dụng để vẽ biểu đồ phân tán của dữ liệu cùng với vị trí của các medoid. Hàm hoạt động như sau:

1. Nhận vào một DataFrame `df` chứa dữ liệu và một DataFrame `medoids` chứa vị trí của các medoid.
2. Tạo một biểu đồ phân tán với các điểm dữ liệu được tô màu theo nhóm medoid mà chúng thuộc về.
3. Vẽ các medoid trên biểu đồ với hình dạng khác biệt (hình vuông) và kích thước lớn hơn để dễ nhận biết.
4. Đặt nhãn cho trục x và y, cũng như thiết lập kích thước phông chữ.
5. Tạo một chú thích (legend) để hiển thị vị trí của từng medoid trên biểu đồ.

```
In [59]: def plot_cluster(df, medoids):
    k = medoids.shape[0]

    fig, ax = plt.subplots(figsize=(8, 6))

    plt.scatter(df.iloc[:, 0], df.iloc[:, 1], marker='o', c=df['medoid'].astype('category'), cmap='cool', s=80, a

    scatter = plt.scatter(medoids.iloc[:, 0], medoids.iloc[:, 1], marker='s', s=200, c=[i for i in range(k)], cma

    ax.set_xlabel(r'x', fontsize=14)
    ax.set_ylabel(r'y', fontsize=14)

    plt.xticks(fontsize=12)
    plt.yticks(fontsize=12)

    legend_labels = [f'medoid {i} : ({medoids.iloc[i, 0]:.2f}, {medoids.iloc[i, 1]:.2f})' for i in range(k)]
    plt.legend(handles=scatter.legend_elements()[0], labels=legend_labels, title='medoids')

    return ax
```

Hàm `kmedian` thực hiện thuật toán K-medians để phân cụm dữ liệu. Hàm hoạt động khá như `kmeans` nhưng code này được điều chỉnh để có thể hiển thị đồ thị phân tán sau mỗi lần cập nhật vị trí của các medoid. Cụ thể:

1. Nhận vào một DataFrame `dset` chứa dữ liệu, số lượng cụm `k`, ngưỡng dừng `tol`, vị trí khởi tạo của các medoid `init_medoids`, và một cờ `show_plot` để quyết định có hiển thị biểu đồ phân tán hay không.
2. Tạo một bản sao của dữ liệu gốc để làm việc và khởi tạo một danh sách `err` để lưu trữ lỗi của mỗi lần lặp.
3. Nếu `init_medoids` không được cung cấp, hàm sẽ khởi tạo ngẫu nhiên các medoid ban đầu.
4. Trong vòng lặp chính, hàm gán mỗi điểm dữ liệu vào medoid gần nhất và tính toán lỗi của lần gán này.
5. Nếu `show_plot` là True, hàm sẽ vẽ biểu đồ phân tán của dữ liệu với vị trí của các medoid sau mỗi lần cập nhật.
6. Cập nhật vị trí của các medoid bằng cách tính giá trị trung vị của các điểm dữ liệu được gán cho mỗi medoid.
7. Kiểm tra điều kiện dừng dựa trên sự thay đổi của lỗi giữa các lần lặp. Nếu sự thay đổi nhỏ hơn hoặc bằng `tol`, vòng lặp sẽ dừng lại.
8. Trả về nhãn của các cụm, lỗi của lần gán cuối cùng, và vị trí cuối cùng của các medoid.

```
In [60]: def kmedian(dset, k=2, tol=1e-4, init_medoids=None, show_plot=True):
    working_dset = dset.copy()
    err = []
    goahead = True

    j = 0
    if init_medoids is None:
        init_medoids = initiate_centroids(k, working_dset)

    medoids = init_medoids

    while goahead:
        working_dset['medoid'], j_err = centroid_assignment(working_dset.iloc[:, :2], medoids)

        err.append(sum(j_err))
        if show_plot is True:
            ax = plot_cluster(working_dset, medoids)
            ax.set_title(f'Iteration {j}')
            plt.show()

        medoids = working_dset.groupby('medoid').agg('median').reset_index(drop=True)

        if j > 0:
            if err[j-1] - err[j] <= tol:
                goahead = False

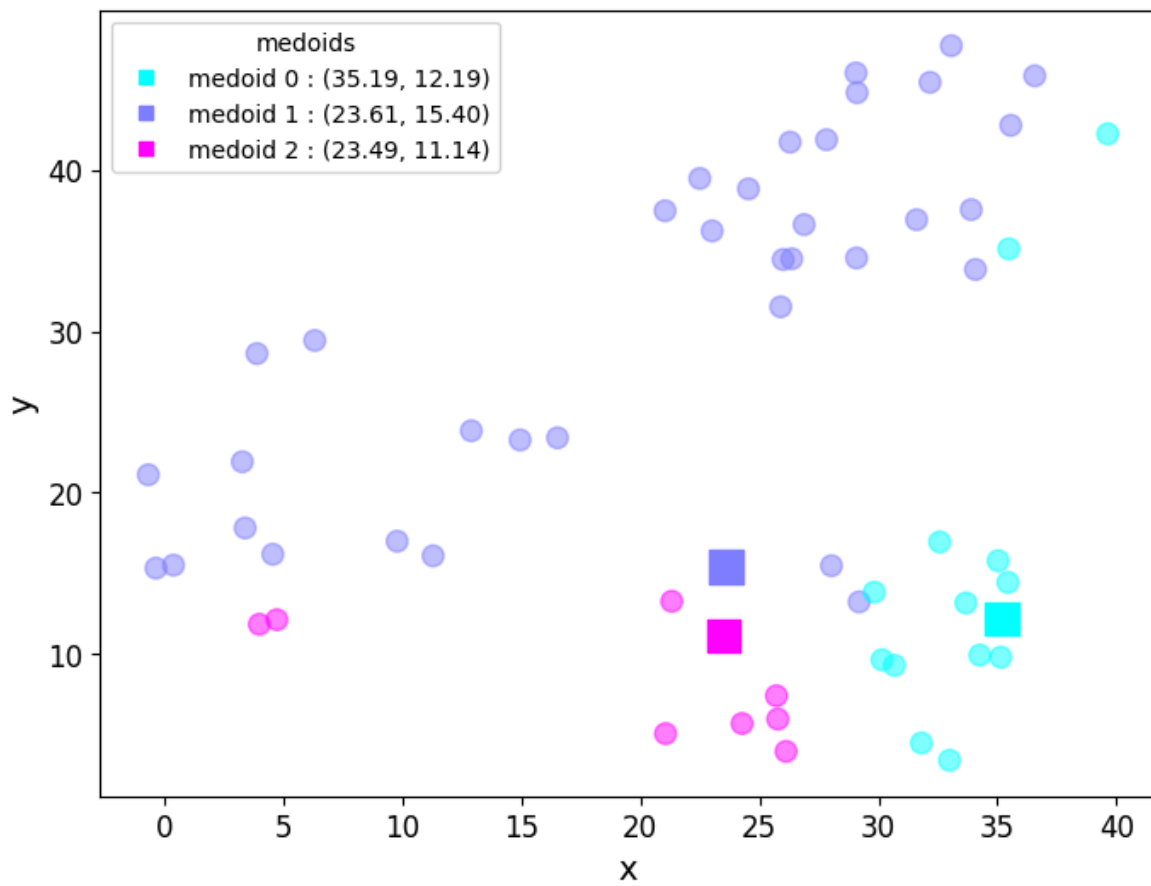
        j += 1

    return working_dset['medoid'], j_err, medoids
```

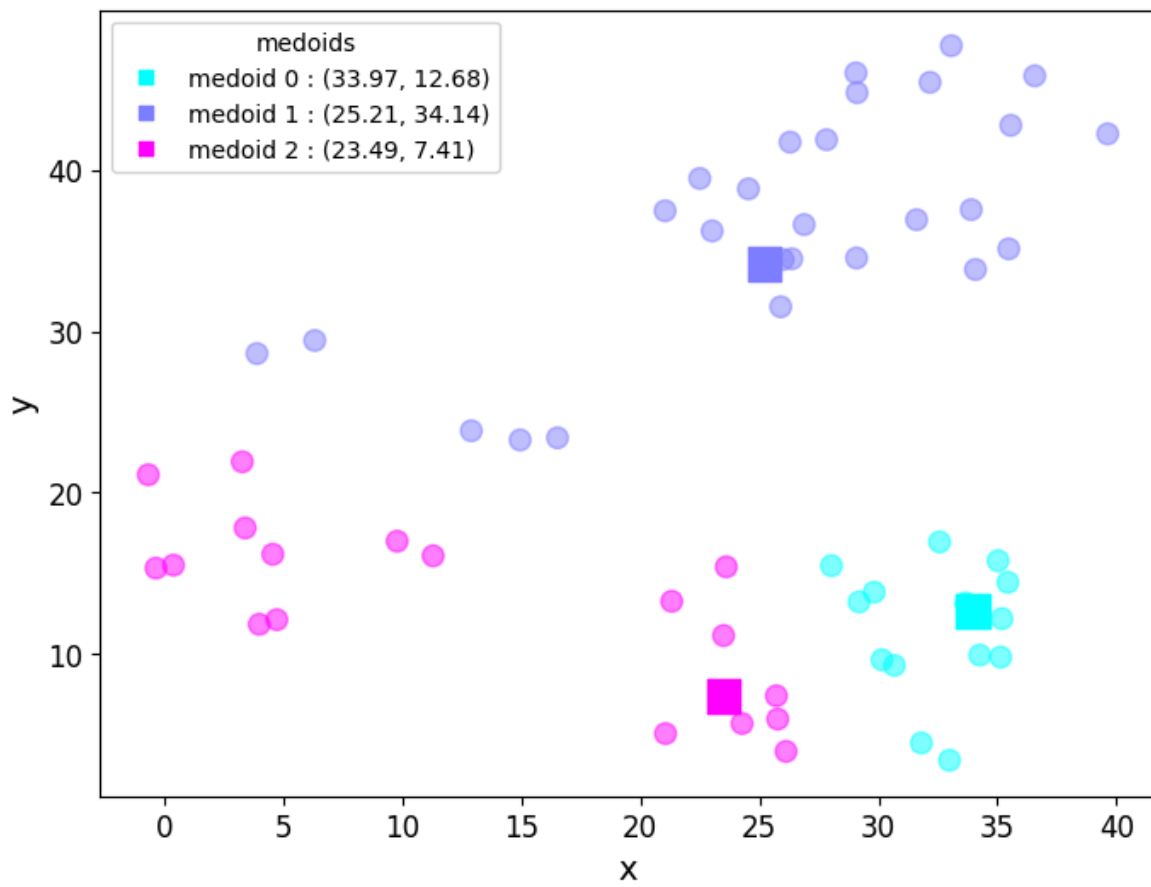
Chạy thử kmedian với k = 3

```
In [61]: np.random.seed(42)
init_medoids=initiate_centroids(3,df)
df['medoid'],df['error'],medoids=kmedian(df[['x','y']],k=3,init_medoids=init_medoids)
```

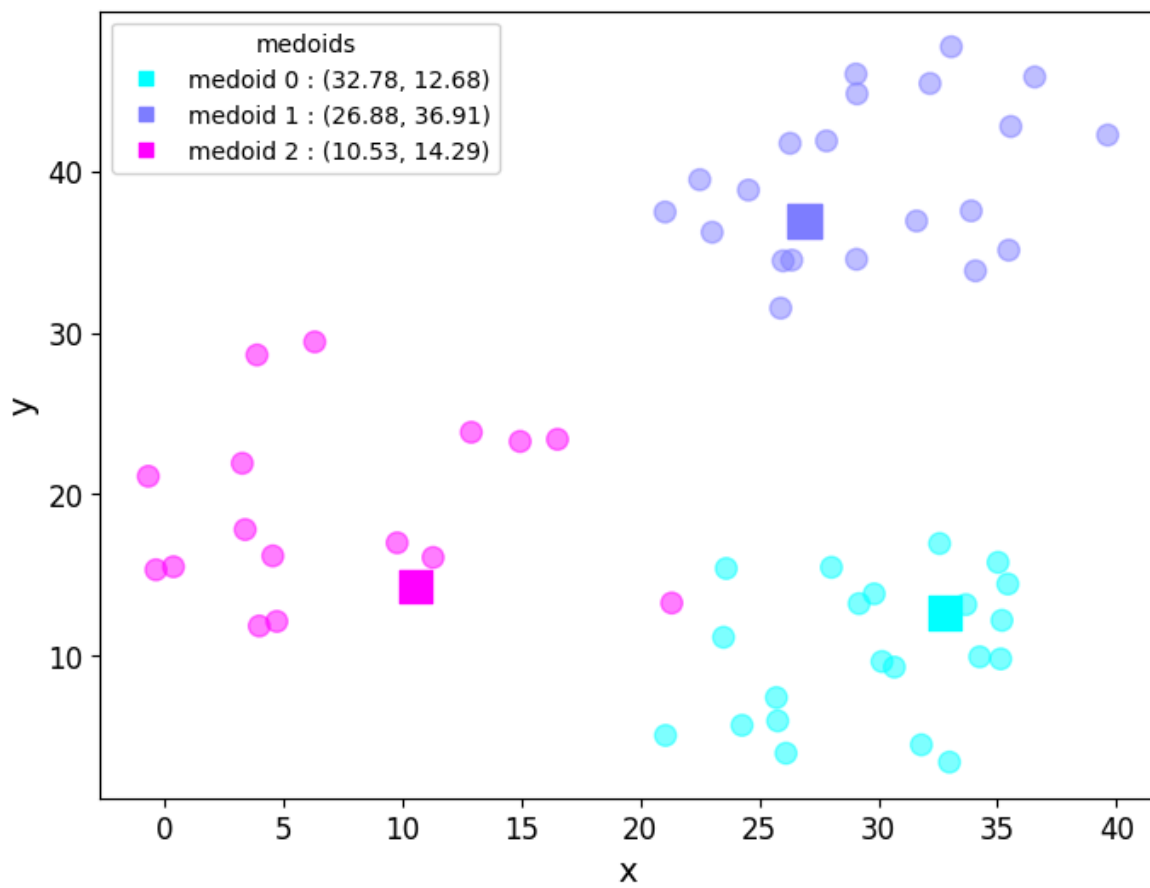

Iteration 0



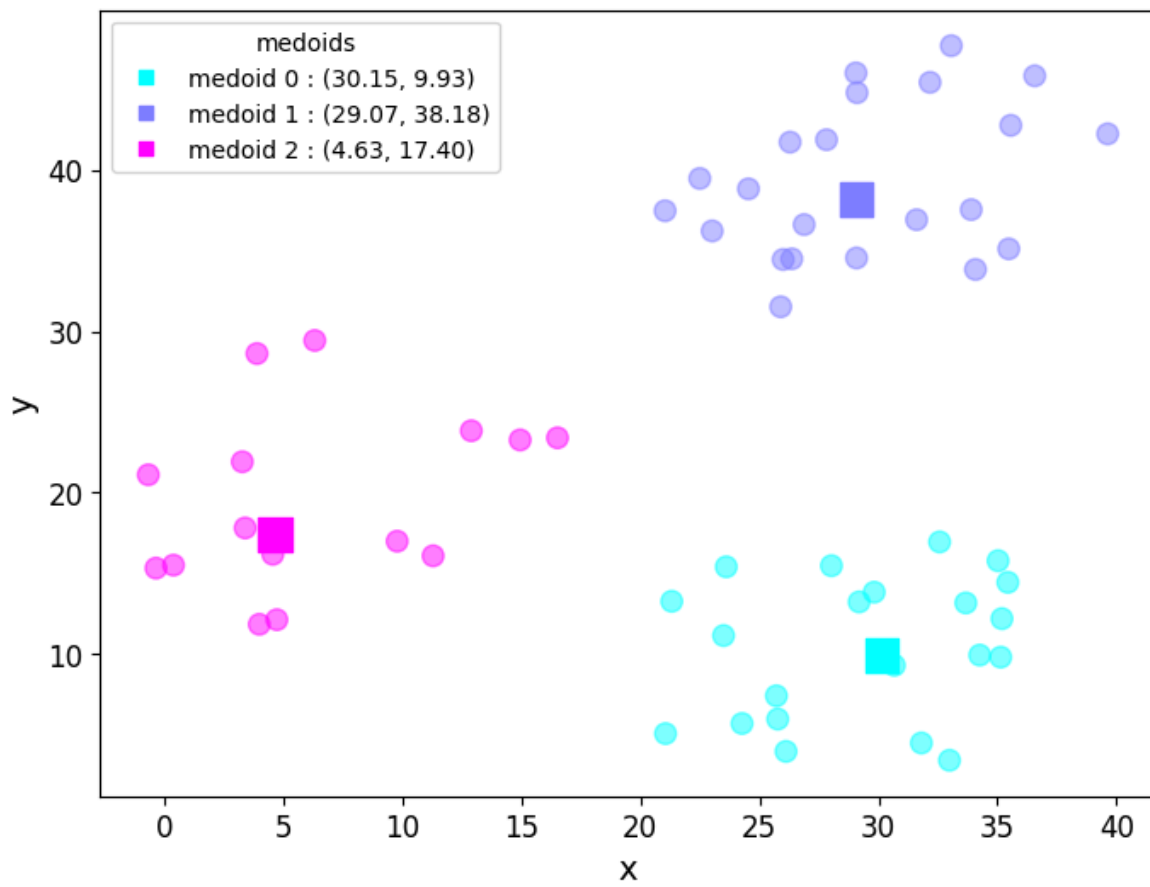
Iteration 1

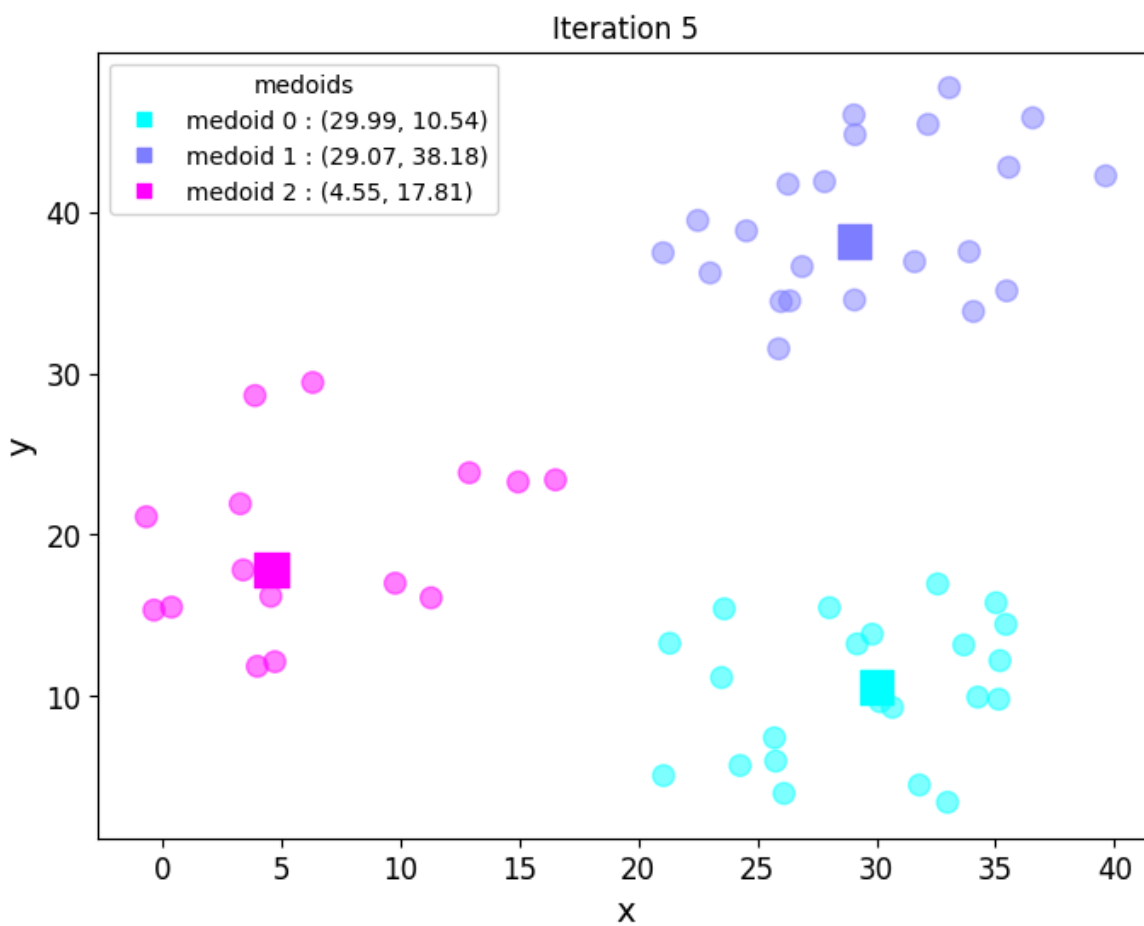
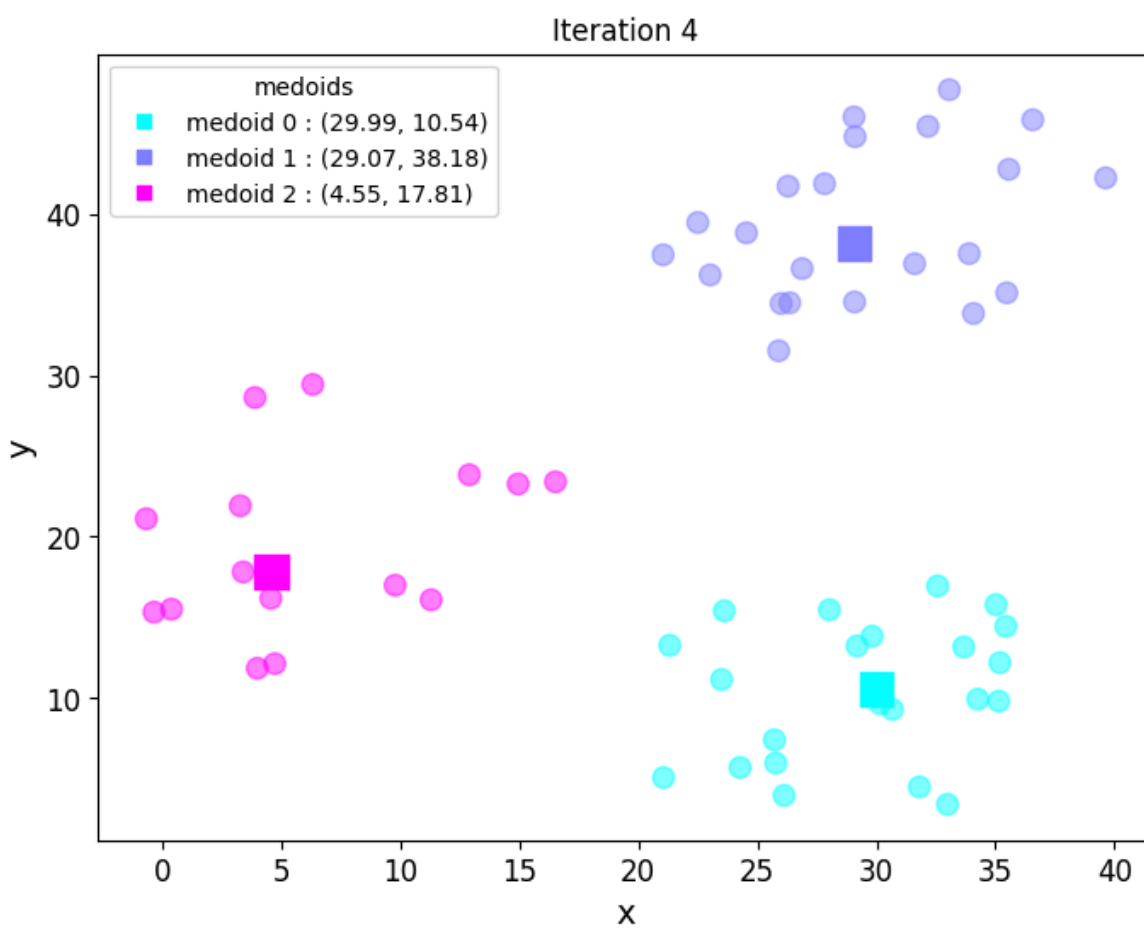


Iteration 2



Iteration 3





Tương tự như kmeans, ta dùng kỹ thuật “elbow” để chỉ ra số cụm tối ưu

```
In [62]: err_total=[]
n=10
df_elbow=blobs[['x','y']]
for i in range(n):
    _,my_errs,_=kmedian(df_elbow,k=i+1,show_plot=False)
    err_total.append(sum(my_errs))

fig,ax=plt.subplots(figsize=(8,6))
```

```
plt.plot(range(1,n+1),err_total,linewidth=3,marker='o')
ax.set_xlabel(r'Number of clusters',fontsize=14)
ax.set_ylabel(r'Total error',fontsize=14)
plt.xticks(fontsize=12)
plt.yticks(fontsize=12)
plt.show()
```

